```
!pip install torch torchvision torchmetrics matplotlib segmentation-models-pytorch onnx onnx-tf tensorflow
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.11.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.26.0+cu128)
Requirement already satisfied: torchmetrics in /usr/local/lib/python3.12/dist-packages (1.9.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: segmentation-models-pytorch in /usr/local/lib/python3.12/dist-packages (0.5.0)
Requirement already satisfied: onnx in /usr/local/lib/python3.12/dist-packages (1.21.0)
Requirement already satisfied: onnx-tf in /usr/local/lib/python3.12/dist-packages (1.6.0)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curan
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cu
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cuf
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cu
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cu
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufil
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufil
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufil
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufil
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupt
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: packaging>17.1 in /usr/local/lib/python3.12/dist-packages (from torchmetrics) (26.2)
Requirement already satisfied: lightning-utilities>=0.15.3 in /usr/local/lib/python3.12/dist-packages (from torchmetrics) (0.15.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: huggingface-hub>=0.24 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (1.17.0)
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (0.7.0)
Requirement already satisfied: timm>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (1.0.27)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (4.67.3)
Requirement already satisfied: protobuf>=4.25.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (5.29.6)
Requirement already satisfied: ml_dtypes>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from onnx) (0.5.4)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from onnx-tf) (6.0.3)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
```

```
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
```

```
import torch
import torchvision
import numpy as np
import matplotlib.pyplot as plt

from torch.utils.data import Dataset, DataLoader, Subset
from torchvision.datasets import OxfordIIITPet
from torchvision import transforms
from PIL import Image

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
IMG_SIZE = 128
```

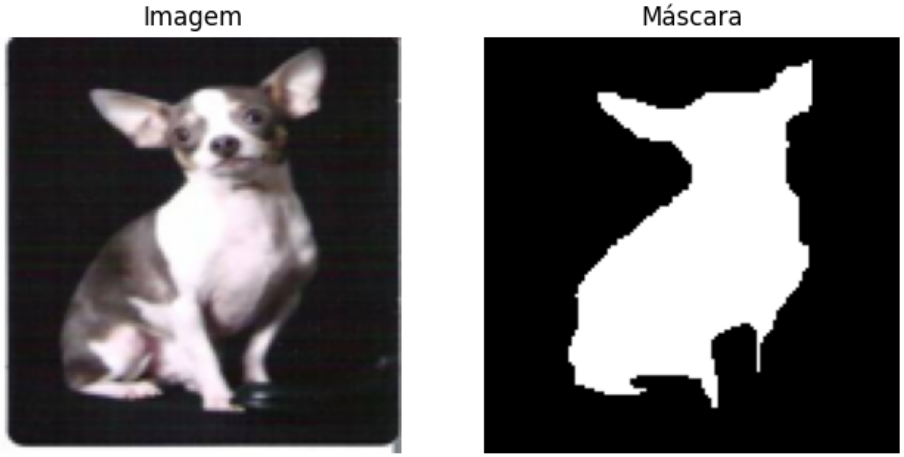
```
class PetSegmentationDataset(Dataset):
    def __init__(self, root, split="trainval", img_size=128, download=True):
        self.dataset = OxfordIIITPet(
            root=root,
            split=split,
            target_types="segmentation",
            download=download
        )
        self.img_size = img_size

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        image, mask = self.dataset[idx]

        image = image.resize((self.img_size, self.img_size))
        mask = mask.resize((self.img_size, self.img_size), resample=Image.NEAREST)

        image = transforms.ToTensor()(image)

        mask = np.array(mask)

        mask = (mask ==  1).astype(np.float32)

        mask = torch.tensor(mask).unsqueeze(0)

        return image, mask
```

```
train_dataset_full = PetSegmentationDataset(root="./data", split="trainval", img_size=IMG_SIZE)
test_dataset_full = PetSegmentationDataset(root="./data", split="test", img_size=IMG_SIZE)

train_dataset = Subset(train_dataset_full, range(1000))
test_dataset = Subset(test_dataset_full, range(200))

train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)

print("Treino:", len(train_dataset))
print("Teste:", len(test_dataset))
```

```
Treino: 1000
Teste: 200
```

```
images, masks = next(iter(train_loader))

print("Imagem:", images.shape)
print("Máscara:", masks.shape)

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.imshow(images[0].permute(1,2,0))
plt.title("Imagem")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(masks[0][0], cmap="gray")
plt.title("Máscara")
plt.axis("off")

plt.show()
```

```
Imagem: torch.Size([8, 3, 128, 128])
Máscara: torch.Size([8, 1, 128, 128])
```




```
import segmentation_models_pytorch as smp

model = smp.Unet(
    encoder_name="resnet18",
    encoder_weights="imagenet",
    in_channels=3,
    classes=1
)

model = model.to(DEVICE)

print("Modelo carregado!")
```

```
Modelo carregado!
```

```
import torch.nn as nn

criterion = nn.BCEWithLogitsLoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-4
)
```

```
def train_epoch(model, loader, optimizer, criterion):
    model.train()

    running_loss = 0
```

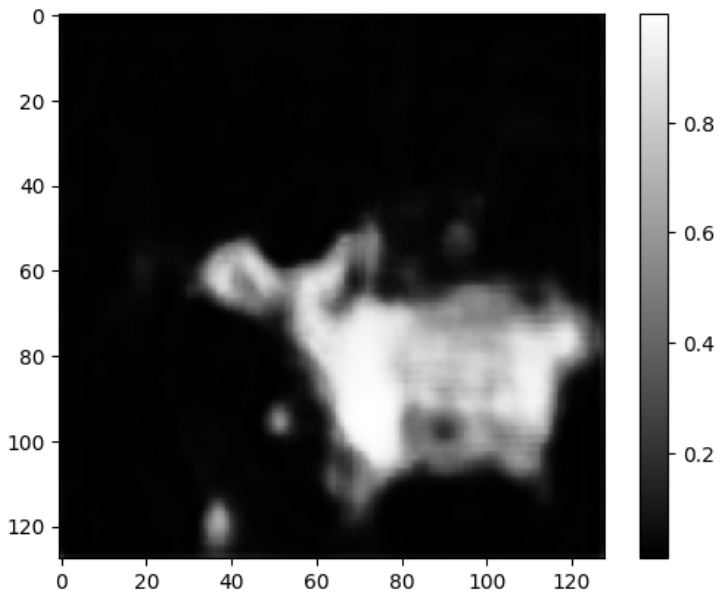
```
    for images, masks in loader:

        images = images.to(DEVICE)
        masks = masks.to(DEVICE)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, masks)

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

    return running_loss / len(loader)
```

```
EPOCHS = 10

for epoch in range(EPOCHS):
    loss = train_epoch(model, train_loader, optimizer, criterion)
    print(f"Epoch {epoch+1}/{EPOCHS} - Loss: {loss:.4f}")
```

```
Epoch 1/10 - Loss: 0.5817
Epoch 2/10 - Loss: 0.3241
Epoch 3/10 - Loss: 0.2405
Epoch 4/10 - Loss: 0.1975
Epoch 5/10 - Loss: 0.1678
Epoch 6/10 - Loss: 0.1486
Epoch 7/10 - Loss: 0.1315
Epoch 8/10 - Loss: 0.1184
Epoch 9/10 - Loss: 0.1074
Epoch 10/10 - Loss: 0.1011
```

```
model.eval()

image, mask = test_dataset[0]

with torch.no_grad():

    pred = model(
        image.unsqueeze(0).to(DEVICE)
    )

    pred = torch.sigmoid(pred)

    pred = pred.cpu().squeeze().numpy()
    plt.imshow(pred, cmap="gray")
```

```
    plt.colorbar()
```

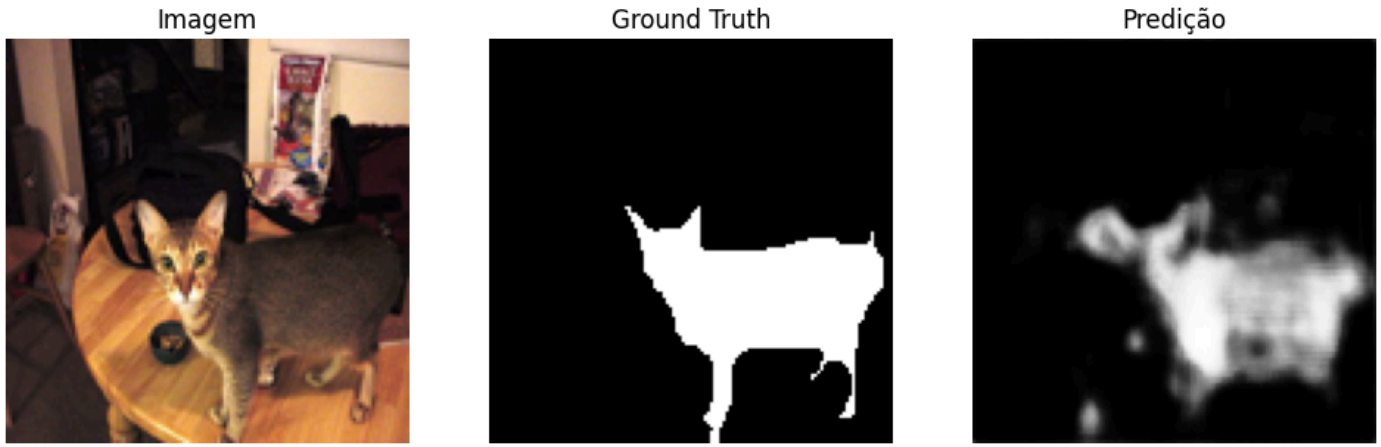



```
from torchmetrics.classification import BinaryJaccardIndex
from torchmetrics.classification import BinaryAccuracy

iou_metric = BinaryJaccardIndex().to(DEVICE)
acc_metric = BinaryAccuracy().to(DEVICE)

model.eval()

iou_metric.reset()
acc_metric.reset()

with torch.no_grad():

    for images, masks in test_loader:

        images = images.to(DEVICE)
        masks = masks.to(DEVICE)

        outputs = model(images)

        outputs = torch.sigmoid(outputs)

        preds = (outputs > 0.5).float()
```

```
            iou_metric.update(preds, masks)
            acc_metric.update(preds, masks)

    iou = iou_metric.compute().item()
    acc = acc_metric.compute().item()

    print(f"IoU: {iou:.4f}")
    print(f"Acurácia: {acc:.4f}")
```

```
IoU: 0.7935
Acurácia: 0.9320
```

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(image.permute(1,2,0))
plt.title("Imagem")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(mask.squeeze(), cmap="gray")
plt.title("Ground Truth")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(pred, cmap="gray")
plt.title("Predição")
plt.axis("off")

plt.savefig("resultado_segmentacao.png", dpi=200, bbox_inches="tight")
plt.show()
```



```
torch.save(model.state_dict(), "unet_pet_segmentation.pth")
```

```
!pip install onnx onnxscript
```

```
Requirement already satisfied: onnx in /usr/local/lib/python3.12/dist-packages (1.21.0)
Requirement already satisfied: onnxscript in /usr/local/lib/python3.12/dist-packages (0.7.0)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.12/dist-packages (from onnx) (2.0.2)
Requirement already satisfied: protobuf>=4.25.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (5.29.6)
Requirement already satisfied: typing_extensions>=4.7.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (4.15.0)
Requirement already satisfied: ml_dtypes>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from onnx) (0.5.4)
Requirement already satisfied: onnx_ir<2,>=0.1.16 in /usr/local/lib/python3.12/dist-packages (from onnxscript) (0.2.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from onnxscript) (26.2)
Requirement already satisfied: sympy>=1.13 in /usr/local/lib/python3.12/dist-packages (from onnx_ir<2,>=0.1.16->onnxscript) (1.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13->onnx_ir<2,>=0.1.16->onnxscript) (1.3.0)
```

```
dummy_input = torch.randn(1, 3, IMG_SIZE, IMG_SIZE).to(DEVICE)

torch.onnx.export(
    model,
    dummy_input,
    "unet_pet_segmentation.onnx",
    input_names=["input"],
    output_names=["output"],
    opset_version=11
)

print("Modelo ONNX exportado!")
```

```
W0607 15:02:08.301000 1542 torch/onnx/_internal/exporter/_compat.py:133] Setting ONNX exporter to use operator set version 18 because the requested opset_
[torch.onnx] Obtain model graph for `Unet([...]` with `torch.export.export(..., strict=False)`...
[torch.onnx] Obtain model graph for `Unet([...]` with `torch.export.export(..., strict=False)`... ✅
[torch.onnx] Run decompositions...
/usr/lib/python3.12/copyreg.py:99: FutureWarning: `isinstance(treespec, LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec) and treespec.is_leaf
  return cls.__new__(cls, *args)
[torch.onnx] Run decompositions... ✅
[torch.onnx] Translate the graph into ONNX...
WARNING:onnxscript.version_converter:The model version conversion is not supported by the onnxscript version converter and fallback is enabled. The model
[torch.onnx] Translate the graph into ONNX... ✅
WARNING:onnxscript.version_converter:Failed to convert the model to the target version 11 using the ONNX C API. The model was not modified
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/onnxscript/version_converter/__init__.py", line 137, in call
    converted_proto = _c_api_utils.call_onnx_api(
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/onnxscript/version_converter/_c_api_utils.py", line 65, in call_onnx_api
    result = func(proto)
             ^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/onnxscript/version_converter/__init__.py", line 132, in _partial_convert_version
    return onnx.version_converter.convert_version(
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/onnx/version_converter.py", line 39, in convert_version
    converted_model_str = C.convert_version(model_str, target_version)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
RuntimeError: /github/workspace/onnx/version_converter/BaseConverter.h:64: adapter_lookup: Assertion `false` failed: No Adapter To Version $17 for Resize
[torch.onnx] Optimize the ONNX graph...
[torch.onnx] Optimize the ONNX graph... ✅
Modelo ONNX exportado!
```

```
torch.save(model.state_dict(), "unet_pet_segmentation.pth")

with open("metricas.txt", "w") as f:
    f.write("IoU: 0.7903\n")
    f.write("Acurácia: 0.9304\n")
```

```
from google.colab import files

files.download("unet_pet_segmentation.pth")
files.download("unet_pet_segmentation.onnx")
```